

Examen SEOC CEAMC

Calculatrice basique autorisée
Ordinateurs et autres portables/mobiles
interdits

Rendre chaque partie sur des feuilles
séparées

Rendre le sujet

Sujet sur 10 pages

Première partie, 12 points

Fonction de hachage

On considère un algorithme de hachage simple sur une chaîne de caractères ASCII (8 bits par caractère). La fonction renvoie un entier non signé sur 64 bits qui correspond aux XOR bit-à-bit successifs de chaque ensemble de 8 caractères de la chaîne. Si le nombre de caractères dans la chaîne n'est pas un multiple de 8, la fonction de hachage se comporte comme si les caractères manquants pour arriver à un multiple de 8 valaient 0. Plus intuitivement, si l'entier 64 bits qui résulte de la fonction de hachage est vu comme un tableau de 8 entiers 8 bits, alors l'entier à l'indice $i \in [0, 8[$ de ce tableau est le XOR de tous les caractères d'index n tel que $(n \bmod 8)$ vaut i . Le déroulement de l'algorithme sur une chaîne de 19 caractères ASCII est donné en Figure 1.

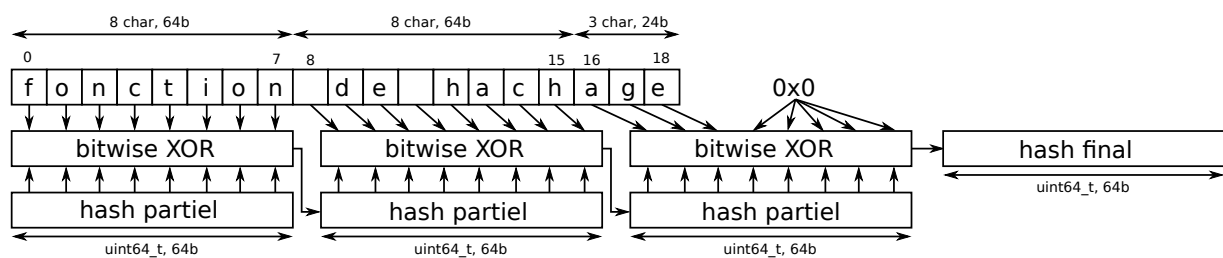


Figure 1: Application de la fonction de hachage à la chaîne "fonction de hachage".

Cet algorithme peut être implémenté en C++ de la façon suivante (ici on incrémente i de 8 en 8 pour décaler chaque caractère du nombre de bits approprié car on travaille sur l'entier 64-bit directement plutôt que sur un tableau de 8 entiers 8-bit):

```
uint64_t my_hash(const std::string & to_hash)
{
    uint64_t hash = 0;
    int i = 0;
    for(char c : to_hash) // Pour chaque caractere de to_hash
    {
        uint64_t shifted_char = ((uint64_t) c) << i;
        hash = hash ^ shifted_char;
        i = (i + 8) % 64; // i dans (0,8,16,24,32,40,48,56)
    }
    return hash;
}
```

Ce code C est ensuite compilé vers le code RISC-V 64-bit suivant (les registres $a0 - a7$ correspondent aux registres $x10 - x17$ vus en cours).

```

.my_hash:
    ld      a5, 0(a0)           // a5 = ptr sur elt courant chaine
    ld      a1, 8(a0)           // a1 = taille de la chaine
    add     a1, a5, a1           // a1 = pointeur apres dernier elt
    beq     a1, a5, .empty_str  // Si chaine vide, saute
    li      a3, 0                // a3 = i = 0
    li      a0, 0                // a0 = hash = 0
.for_loop:
    lbu     a4, 0(a5)           // c = *ptr
    addi    a2, a3, 8            // a2 = i + 8
    addi    a5, a5, 1            // ptr++
    sll     a4, a4, a3           // c = c << i
    xor     a0, a0, a4           // hash = hash ^ c
    andi    a3, a2, 63           // i = (i + 8) & 63 <=> (i + 8) % 64
    bne     a1, a5, .for_loop    // Si reste des elts, boucle continue
    ret                                           // Boucle terminee, on renvoie hash (a0)
.empty_str:
    li      a0, 0                // Chaine vide, on renvoie 0 (a0)
    ret

```

1 Ordonnancement

Question 1.1: Rappelez les différents types de dépendances de données via les registres et ce qu'elles expriment.

Question 1.2: Existe-t-il d'autres types de dépendances de données qui pourraient limiter la performance dans ce programme ? En général ?

Question 1.3: Mettre en lumière les dépendances de données via les registres présentes dans la boucle du code assembleur (de `.for_loop` à `.empty_str`). Attention à ne pas oublier les dépendances entre plusieurs itérations de la boucle.

Question 1.4: En considérant toutes les dépendances de données via les registres mises en lumière à la question précédente, et en considérant que chaque instruction requiert un seul cycle pour s'exécuter, quel est le parallélisme d'instructions présent dans la boucle de la fonction, en instructions par cycle (IPC), en admettant que la chaîne de caractères soit grande. Attention, on considère ici que deux instructions dépendantes ne peuvent **jamais** être ordonnancées dans le même cycle, même si certains designs pourraient se le permettre. On ignorera de plus les dépendances de contrôle (i.e., prédiction de branchement parfaite).

Question 1.5: Rappelez brièvement le principe du renommage de registres et ses avantages, les structures matérielles requises pour l'implémenter et leur(s) rôle(s).

Question 1.6 (Bonus): Même question que 1.4, mais en ignorant les dépendances de données via les registres que le renommage de registres permet d'ignorer (on considère qu'on a un nombre infini de registres physiques à notre disposition).

2 Optimisation microarchitecturale via le renommage de registres

On se place dans le contexte d'un jeu d'instruction définissant 31 registres généralistes 64-bit (`R1` à `R31`). Le jeu d'instruction définit de plus un registre `R0` toujours `0x0`, `R0`.

On possède un processeur implémentant cette architecture via une microarchitecture à exécution dans le désordre. Les structures nécessaires à l'exécution correcte et efficace des instructions sont donc déjà présentes : *Rename Map Table*, *Scoreboard*, *Active List*, *Free List*, *Physical Register File* (fichier de registres) *Reorder Buffer*, *Instruction Queue* (ordonnanceur) et *Load/Store Queue*.

Dans cette microarchitecture, le fichier de registres possède 127 registres physiques $p1$ à $p127$, auquel on ajoute un unique registre câblé à $0x0$, $p0$ afin d'implémenter le registre logique $R0$. En particulier, une instruction utilisant $R0$ comme opérande (source ou destination) renommera toujours $R0$ par $p0$. Cela signifie notamment que $p0$ ne fait pas partie de la *Free List*, et que le registre $p0$ est toujours prêt dans le *Scoreboard*, car sa valeur courante est toujours $0x0$.

On souhaite améliorer la performance de la microarchitecture existante en tirant parti du renommage de registre. En particulier, on remarque qu'il est possible de calculer le résultat de certaines instructions sans les exécuter, mais en manipulant l'association des registres physiques aux registres logiques.

On remarque que les instructions pour lesquelles on peut prouver que le résultat généré vaut $0x0$ simplement via l'instruction et le nom de ses opérandes (registres architecturaux) peut "s'éliminer" (i.e., n'a pas besoin d'être ordonnancée et exécutée) à l'étage de renommage en renommant le registre logique de destination par $p0$. Quelques exemples où renommer $R1$ par $p0$ permet d'exécuter l'instruction au renommage :

```
xor  R1, R2, R2    # R1 = R2 xor R2 = 0x0
and  R1, R2, R0    # R1 = R2 and 0x0 = 0x0
or   R1, R0, R0    # R1 = 0x0 or 0x0 = 0x0
```

Question 2.1 : Quel(s) avantage(s) aura-t-on à éliminer les instructions que l'on peut prouver comme produisant toujours $0x0$ dans l'étage de renommage ?

Question 2.2 : Est-ce qu'une instruction éliminée peut complètement disparaître de toutes les structures du pipeline au renommage ? On pourra par exemple considérer la suite d'instructions suivante :

```
A:
add  R2, R4, R5    # R2 = R4 + R5
cbnz R2, B        # Saut vers B si R2 != 0
xor  R3, R2, R2    # R3 = R2 xor R2 = 0x0; elim
...
B:
sub  R1, R2, R3    # R1 = R2 + R3
```

Est-il alors possible de faire complètement disparaître *xor R1, R2, R2* du pipeline au renommage ?

Question 2.3 (Bonus) : Est-ce qu'implémenter cette élimination dans notre microarchitecture requiert un support matériel particulier (par ex., nouvelles structures, nouveaux champs dans des structures existantes, nouveaux circuits, etc.) ? On pourra notamment chercher à répondre aux questions suivantes ¹ :

- Quel impact sur l'algorithme assurant le recyclage des registres physiques ?
- Quel impact sur la réparation des structures de l'étage de renommage lors d'un vidage de pipeline ?

¹Les questions sont données à titre indicatif afin de guider l'élève. L'objectif est cependant de fournir une réponse à la Question 2.3 dans son entièreté.

Seconde partie, 12 points

1 Étude de performance du calcul d'une image multi-résolution

En synthèse d'image une image multi-résolution sert à afficher une texture vue selon différents points de vue. Une image multi-résolution (dite MipMap dans la suite) est constituée de l'image originale et de différentes échelles de cette image, les échelles étant des puissances de 2. Une MipMap isotropique est constituée des images d'échelles identiques en horizontal et vertical. Une MipMap anisotropique est constituée d'échelles différentes en horizontal et vertical. Dans la suite nous ne considérons que les MipMap isotropiques.



Exemple d'image multirésolution isotropique

Le calcul de chaque niveau de résolution se fait récursivement à partir du précédent par une décimation précédée d'un filtre décimateur. Le filtre décimateur est ici un filtre d'image de taille 3×3 , qui est le produit d'un filtre 1D d'ordre 3, f . Il est possible de réaliser le filtre soit en une passe par l'application d'une convolution 2D, soit en deux passes par les applications successives du filtre 1D f en horizontal puis en vertical.

1.1 Notations

Un pixel de coordonnée (x, y) de l'image I est noté $I(x, y)$.

Le pixel de coordonnée $I(0, 0)$, est en haut à gauche. Une image I a une taille $I_x \times I_y$ et on notera une zone $z = z_x \times z_y$ de l'image I de largeur de z_x pixels et une hauteur de z_y pixels.

Une boucle 2D sur la zone z est indexée à partir de $(0, 0)$ et va jusqu'à $(z_x - 1, z_y - 1)$ inclus.

La convolution d'une image I par un filtre F est noté $O = I \otimes F$

1.2 Filtrage en deux passe

L'image filtrée R est produite à partir de l'image d'entrée I . Le filtrage en deux passes commence par produire une première image R_H , de taille $I_x \times I_y$, à l'aide d'un filtre 1D f horizontal. Ensuite l'image R est obtenue à partir de R_H en appliquant un filtre vertical 1D. L'image R est aussi de taille $I_x \times I_y$.

Le filtrage horizontal d'une image se fait en appliquant le filtre 1D sur tous les pixels de l'entrée. Concrètement, pour chaque pixel de coordonnée (i, j) de l'image de sortie, on calcule la valeur $R_H(i, j)$ telle que :

$$R_H(i, j) = \sum_{k=-1}^1 I(i + k, j) * f(k + 1)$$

Par souci de simplicité, on ne tient pas compte des bords pour lesquels $i + k$ est soit négatif soit trop grand.

Pour la deuxième passe, on calcule le pixel $R(i, j)$ à partir de R_H de la façon suivante :

$$R(i, j) = \sum_{l=-1}^1 R_H(i, j + l) * f(l + 1)$$

Les coefficients de f sont :

$$f = \left(\frac{1}{4}, \frac{1}{2}, \frac{1}{4}\right)$$

1.3 Filtrage en une passe

Une image O est le filtrage 2D d'une image d'entrée I par application du filtre de convolution 2D F et se calcule selon cette équation :

$$O(i, j) = \sum_{k=-1}^1 \sum_{l=-1}^1 I(i + k, j + l) * F(k + 1, l + 1)$$

O et I sont des matrices de tailles $t_x \times t_y$ et F est une matrice de taille 3×3 .

Les coefficients de F sont :

$$F = \begin{pmatrix} \frac{1}{16} & \frac{1}{8} & \frac{1}{16} \\ \frac{1}{8} & \frac{1}{4} & \frac{1}{8} \\ \frac{1}{16} & \frac{1}{8} & \frac{1}{16} \end{pmatrix}$$

1.4 Construction itérative d'une MipMap

La MipMap P est constituée des différents niveaux de résolution P_r . P_0 est l'image originale de taille $t_x \times t_y$. Pour $r \geq 1$, chaque niveau P_r , de taille $\frac{t_x}{2^r} \times \frac{t_y}{2^r}$ est produit à partir du niveau P_{r-1} précédent par filtrage puis décimation d'un facteur 2 :

$$P_r^f = P_{r-1} \circledast F$$

$$P_r = \downarrow P_r^f$$

La décimation $\downarrow$ consiste à ne prendre qu'un pixel sur deux :

$$P_{rx} \times P_{ry} = \frac{P_{r-1x}}{2} \times \frac{P_{r-1y}}{2} \quad \text{et} \quad P_r(i, j) = P_r^f(2 * i, 2 * j)$$

Dans la suite on considère des images de taille 1024×1024 , et le niveau de résolution maximum vaut 5.

1.5 Etude de performance

On ne tiendra pas compte des opérations nécessaires pour les calculs d'indices. On ne s'intéresse qu'aux opérations sur les données.

▷ **Question 1 :**

Pour une image de taille $t_x \times t_y$, donner le nombre d'additions et de multiplications pour appliquer le filtrage en deux passes.

▷ **Question 2 :**

Même question pour le filtrage en une passe.

L'analyse de l'algorithme montre qu'il est en fait possible d'appliquer simultanément la décimation et le filtrage en une passe. Il suffit de calculer :

$$P_r(i, j) = \sum_{k=-1}^1 \sum_{l=-1}^1 P_{r-1}(2 * i + k, 2 * j + l) * F(k + 1, l + 1)$$

▷ **Question 3 :**

Même question mais pour le filtrage et décimation simultané.

1.6 Etude des accès mémoire

Dans la suite on considère une image P_0 de taille 1024×1024 , et le niveau de résolution maximum vaut 5. Un pixel $I(i, j)$ est à l'adresse $i + j * I_x$ et dans le pseudo-code il est noté par $I(i, j)$.

Considérons un cache dont les caractéristiques sont les suivantes :

Taille	32 KOctet
Ligne	64 Octet
Associatif par groupe	16 voies

Nous allons étudier le nombre de défauts de cache pour calculer un niveau de résolution à partir du précédent. On commencera par étudier le calcul de P_1 à partir de P_0 puis nous généraliserons.

1.6.1 Algorithme avec filtrage en deux passes

On calcule P_r à partir de P_s (c'est à dire $s = r - 1$) et $P_{rx} \times P_{ry} = \frac{P_{sx}}{2} \times \frac{P_{sy}}{2}$.

La première passe qui produit P_{s-H} par filtrage horizontal de P_s est implantée de la façon suivante

```
for(j=0; j<Psy; j++)
  for(i=0; i<Psx; i++)
    acc = 0
    for(k=-1; k<2; k++)
```

```

    acc += Ps(i+k, j)*f(k+1)
    Ps_H(i,j) = acc

```

La seconde passe qui produit P_{s_V} par filtrage vertical de P_{s_H} est implantée de la façon suivante :

```

for(j=0; j<Psy; j++)
    for(i=0; i<Psx; i++)
        acc = 0
        for(l=-1; l<2; l++)
            acc += Ps_H[i, j+l]*f(l+1)
        Ps_V(i,j) = acc

```

La décimation qui produit P_r à partir de P_{s_V} est implémentée par l'algorithme suivant :

```

for(j=0; j<Pry; j++)
    for(i=0; i<Prx; i++)
        Pr(i,j) = Ps_V(2*i, 2*j)

```

▷ **Question 4 :**

Pour le calcul de P_1 à partir de P_0 , donner le nombre de défauts de cache pour chacune des parties de l'algorithme :

- Passe horizontale
- Passe verticale
- Décimation

▷ **Question 5 :**

Après avoir constaté que l'on peut permuter les boucles en i et j de chacun des algorithmes, sans que cela ne modifie le résultat, donner les défauts de cache pour les deux passes et la décimation.

1.6.2 Algorithme avec filtrage en une passe

Le filtrage en une passe qui calcule $P_{s_c} = P_s \circledast F$ est implanté de la façon suivante :

```

for(j=0; j<Psy; j++)
    for(i=0; i<Psx; i++)
        acc = 0
        for(k=-1; k<2; k++)
            for(l=-1; l<2; l++)
                acc += Ps(i+k, j+l)*F(k+1, l+1)
        Ps_c(i, j) = acc

```

▷ **Question 6 :**

Calculer le nombre de défauts de cache du filtrage en une passe

▷ **Question 7 :**

En observant que l'on peut permuter les deux premières boucles, donner le nombre de défauts de cache après cette permutation.

1.6.3 Algorithme avec filtrage et décimation simultanés

L'algorithme de filtrage et décimation simultanés est implanté de la façon suivante

```
for(j=0; j<Prx; j++)
  for(i=0; i<Pry; i++)
    acc = 0
    for(k=-1; k<2; k++)
      for(l=-1; l<2; l++)
        acc += Ps(2*i+k, 2*j+l)*F(k+1, l+1)
    Pr(i,j)= acc
```

▷ **Question 8 :**

Calculer le nombre de défauts de cache de ce filtrage.

▷ **Question 9 :**

En observant que l'on peut permuter les deux premières boucles, donner le nombre de défauts de cache après cette permutation.

1.6.4 Bilan

▷ **Question 10 :**

- Quelle est la version de l'algorithme qui permet de minimiser le nombre de défauts de cache ?
- Quelle est la version de l'algorithme qui permet de minimiser le nombre d'opérations arithmétiques ?

1.7 Calcul de précision

Rappel : pour la notation en virgule fixe, on note le format d'un nombre sous la forme (n, e, v) , avec n le nombre de bits total, e le nombre de bits à gauche de la virgule (partie entière), et v , le nombre de bits à droite (partie décimale). On notera $\lfloor a \rfloor_{(n,e,v)}$ le passage du format de a au format (n, e, v) par troncature.

Dans la suite, tous les nombres sont des entiers non-signés.

Toutes les données d'un tableau (image), ou d'un vecteur sont représentées selon un même format.

Les données des images sont des entiers sur 8 bits.

1.7.1 Algorithme en deux passes

▷ **Question 11 :**

Quel est le format des coefficients du filtre f ?

▷ **Question 12 :**

En reprenant la notation de la section 1.4, après la première passe, quel sont les format des données de P_{s_H} et de P_{s_V} si on souhaite conserver toute la précision des calculs ?

▷ **Question 13 :**

Comme la pyramide doit être stockée en mémoire dans un tableau de mots de 8 bits, quels sont les bits de P_{s_V} à conserver ?

1.7.2 Algorithme en une passe

▷ **Question 14 :**

Quel est le format des coefficients du filtre F ?

▷ **Question 15 :**

Pour cette unique passe, quel est le format de $O(i, j)$ et quels sont les 8 bits à conserver après le calcul de l'accumulation ?

1.7.3 Bilan

▷ **Question 16 :**

De façon qualitative, quel est l'algorithme qui a le bruit de calcul le plus faible ?